7.5 The Java Collections Framework

Java provides many data structure interfaces and classes, which together form the ***Java Collections Framework***. This framework, which is part of the java.util package, includes versions of several of the data structures discussed in this book, some of which we have already discussed and others of which we will discuss later in this book. The root interface in the Java collections framework is named Collection. This is a general interface for any data structure, such as a list, that represents a collection of elements. The Collection interface includes many methods, including some we have already seen (e.g., size( ), isEmpty( ), iterator( )). It is a superinterface for other interfaces in the Java Collections Framework that can hold elements, including the java.util interfaces Deque, List, and Queue, and other subinterfaces discussed later in this book, including Set (Section 10.5.1) and Map (Section 10.1).

The Java Collections Framework also includes concrete classes implementing various interfaces with a combination of properties and underlying representations. We summarize but a few of those classes in Table 7.3. For each, we denote which of the Queue, Deque, or List interfaces are implemented (possibly several). We also discuss several behavioral properties. Some classes enforce, or allow, a fixed capacity limit. Robust classes provide support for ***concurrency***, allowing multiple processes to share use of a data structure in a thread-safe manner. If the structure is designated as ***blocking***, a call to retrieve an element from an empty collection waits until some other process inserts an element. Similarly, a call to insert into a full blocking structure must wait until room becomes available.

	Interfaces			**Properties**			**Storage**	
Class	Queue	Deque	List	Capacity Limit	Thread-Safe	Blocking	Array	Linked List
ArrayBlockingQueue	✓			✓	✓	✓	✓	
LinkedBlockingQueue	✓			✓	✓	✓		✓
ConcurrentLinkedQueue	✓				✓		✓	
ArrayDeque	✓	✓					✓	
LinkedBlockingDeque	✓	✓		✓	✓	✓		✓
ConcurrentLinkedDeque	✓	✓			✓			✓
ArrayList			✓				✓	
LinkedList	✓	✓	✓					✓

Table 7.3: Several classes in the Java Collections Framework.

7.5.1 List Iterators in Java

The java.util.LinkedList class does not expose a position concept to users in its API, as we do in our positional list ADT. Instead, the preferred way to access and update a LinkedList object in Java, without using indices, is to use a ListIterator that is returned by the list's listIterator( ) method. Such an iterator provides forward and backward traversal methods as well as local update methods. It views its current position as being before the first element, between two elements, or after the last element. That is, it uses a list ***cursor***, much like a screen cursor is viewed as being located between two characters on a screen. Specifically, the java.util.ListIterator interface includes the following methods:

add(e): Adds the element e at the current position of the iterator.

hasNext( ): Returns true if there is an element after the current position of the iterator.

hasPrevious( ): Returns true if there is an element before the current position of the iterator.

previous( ): Returns the element e before the current position and sets the current position to be before e.

next( ): Returns the element e after the current position and sets the current position to be after e.

nextIndex( ): Returns the index of the next element.

previousIndex( ): Returns the index of the previous element.

remove( ): Removes the element returned by the most recent next or previous operation.

set(e): Replaces the element returned by the most recent call to the next or previous operation with e.

It is risky to use multiple iterators over the same list while modifying its contents. If insertions, deletions, or replacements are required at multiple "places" in a list, it is safer to use positions to specify these locations. But the java.util.LinkedList class does not expose its position objects to the user. So, to avoid the risks of modifying a list that has created multiple iterators, the iterators have a "fail-fast" feature that invalidates such an iterator if its underlying collection is modified unexpectedly. For example, if a java.util.LinkedList object L has returned five different iterators and one of them modifies L, a ConcurrentModificationException is thrown if any of the other four is subsequently used. That is, Java allows many list iterators to be traversing a linked list L at the same time, but if one of them modifies L (using an add, set, or remove method), then all the other iterators for L become invalid. Likewise, if L is modified by one of its own update methods, then all existing iterators for L immediately become invalid.

7.5.2 Comparison to Our Positional List ADT

Java provides functionality similar to our array list and positional lists ADT in the java.util.List interface, which is implemented with an array in java.util.ArrayList and with a linked list in java.util.LinkedList.

Moreover, Java uses iterators to achieve a functionality similar to what our positional list ADT derives from positions. Table 7.4 shows corresponding methods between our (array and positional) list ADTs and the java.util interfaces List and ListIterator interfaces, with notes about their implementations in the java.util classes ArrayList and LinkedList.

Positional List ADT Method	**java.util.List Method**	**ListIterator Method**	**Notes**
size( )	size( )		$O(1)$ time
isEmpty( )	isEmpty( )		$O(1)$ time
	get(i)		A is $O(1)$, L is $O(\min\{i, n-i\})$
first( )	listIterator( )		first element is next
last( )	listIterator(size( ))		last element is previous
before(p)		previous( )	$O(1)$ time
after(p)		next( )	$O(1)$ time
set(p, e)		set(e)	$O(1)$ time
	set(i, e)		A is $O(1)$, L is $O(\min\{i, n-i\})$
	add(i, e)		$O(n)$ time
addFirst(e)	add(0, e)		A is $O(n)$, L is $O(1)$
addFirst(e)	addFirst(e)		only exists in L, $O(1)$
addLast(e)	add(e)		$O(1)$ time
addLast(e)	addLast(e)		only exists in L, $O(1)$
addAfter(p, e)		add(e)	insertion is at cursor; A is $O(n)$, L is $O(1)$
addBefore(p, e)		add(e)	insertion is at cursor; A is $O(n)$, L is $O(1)$
remove(p)		remove( )	deletion is at cursor; A is $O(n)$, L is $O(1)$
	remove(i)		A is $O(1)$, L is $O(\min\{i, n-i\})$

Table 7.4: Correspondences between methods in our positional list ADT and the java.util interfaces List and ListIterator. We use A and L as abbreviations for java.util.ArrayList and java.util.LinkedList (or their running times).

7.5.3 List-Based Algorithms in the Java Collections Framework

In addition to the classes that are provided in the Java Collections Framework, there are a number of simple algorithms that it provides as well. These algorithms are implemented as static methods in the java.util.Collections class (not to be confused with the java.util.Collection interface) and they include the following methods:

copy(L_{dest}, L_{src}): Copies all elements of the L_{src} list into corresponding indices of the L_{dest} list.

disjoint(C, D): Returns a boolean value indicating whether the collections C and D are disjoint.

fill(L, e): Replaces each element of the list L with element e.

frequency(C, e): Returns the number of elements in the collection C that are equal to e.

max(C): Returns the maximum element in the collection C, based on the natural ordering of its elements.

min(C): Returns the minimum element in the collection C, based on the natural ordering of its elements.

replaceAll(L, e, f): Replaces each element in L that is equal to e with element f.

reverse(L): Reverses the ordering of elements in the list L.

rotate(L, d): Rotates the elements in the list L by the distance d (which can be negative), in a circular fashion.

shuffle(L): Pseudorandomly permutes the ordering of the elements in the list L.

sort(L): Sorts the list L, using the natural ordering of its elements.

swap(L, i, j): Swap the elements at indices i and j of list L.

Converting Lists into Arrays

Lists are a beautiful concept and they can be applied in a number of different contexts, but there are some instances where it would be useful if we could treat a list like an array. Fortunately, the java.util.Collection interface includes the following helpful methods for generating an array that has the same elements as the given collection:

toArray( ): Returns an array of elements of type Object containing all the elements in this collection.

toArray(*A*): Returns an array of elements of the same element type as *A* containing all the elements in this collection.

If the collection is a list, then the returned array will have its elements stored in the same order as that of the original list. Thus, if we have a useful array-based method that we want to use on a list or other type of collection, then we can do so by simply using that collection's toArray( ) method to produce an array representation of that collection.

Converting Arrays into Lists

In a similar vein, it is often useful to be able to convert an array into an equivalent list. Fortunately, the java.util.Arrays class includes the following method:

asList(*A*): Returns a list representation of the array *A*, with the same element type as the elements of *A*.

The list returned by this method uses the array *A* as its internal representation for the list. So this list is guaranteed to be an array-based list and any changes made to it will automatically be reflected in *A*. Because of these types of side effects, use of the asList method should always be done with caution, so as to avoid unintended consequences. But, used with care, this method can often save us a lot of work. For instance, the following code fragment could be used to randomly shuffle an array of Integer objects, arr:

```
Integer[ ] arr = {1, 2, 3, 4, 5, 6, 7, 8};          // allowed by autoboxing
List<Integer> listArr = Arrays.asList(arr);
Collections.shuffle(listArr);          // this has side effect of shuffling arr
```

It is worth noting that the array *A* sent to the asList method should be a reference type (hence, our use of Integer rather than **int** in the above example). This is because the List interface is generic, and requires that the element type be an object.